

Diplomski studij

**Informacijska i komunikacijska
tehnologija**

Računarstvo

Telekomunikacije i informatika

Obradba informacija

Računalno inženjerstvo

Internet stvari

E-Crkva

Projekt

Dino Hodžić Tumpić

Marcel Macukić

Filip Perušić

Vito Stanić

Marijan Tadijal

Ak.g. 2025./2026.

Sadržaj

1. Uvod.....	3
2. Opis rješenja.....	3
3. IoT platforma.....	4
4. Korisničke aplikacije.....	4

1. Uvod

Crkava i kapela ima mnogo i u gradovima i po selima gdje može biti teško naći koga tko bi zvonio svakih 15, 30, 45 i 60 minuta i nekoga kome bi vjerovali da upravlja zvonikom. Pomoću ovog sustava bi se sve elektroničkim zvonom moglo upravljati udaljeno, te bi mogli pomoću mikrofona provjeriti da li zvono ispravno radi i nalaze li se ljudi trenutno unutar crkve. Različiti obrasci zvonjave se mogu postaviti na razna vremena (npr. dulje za puni sat, kraće za razmak od 15 minuta)

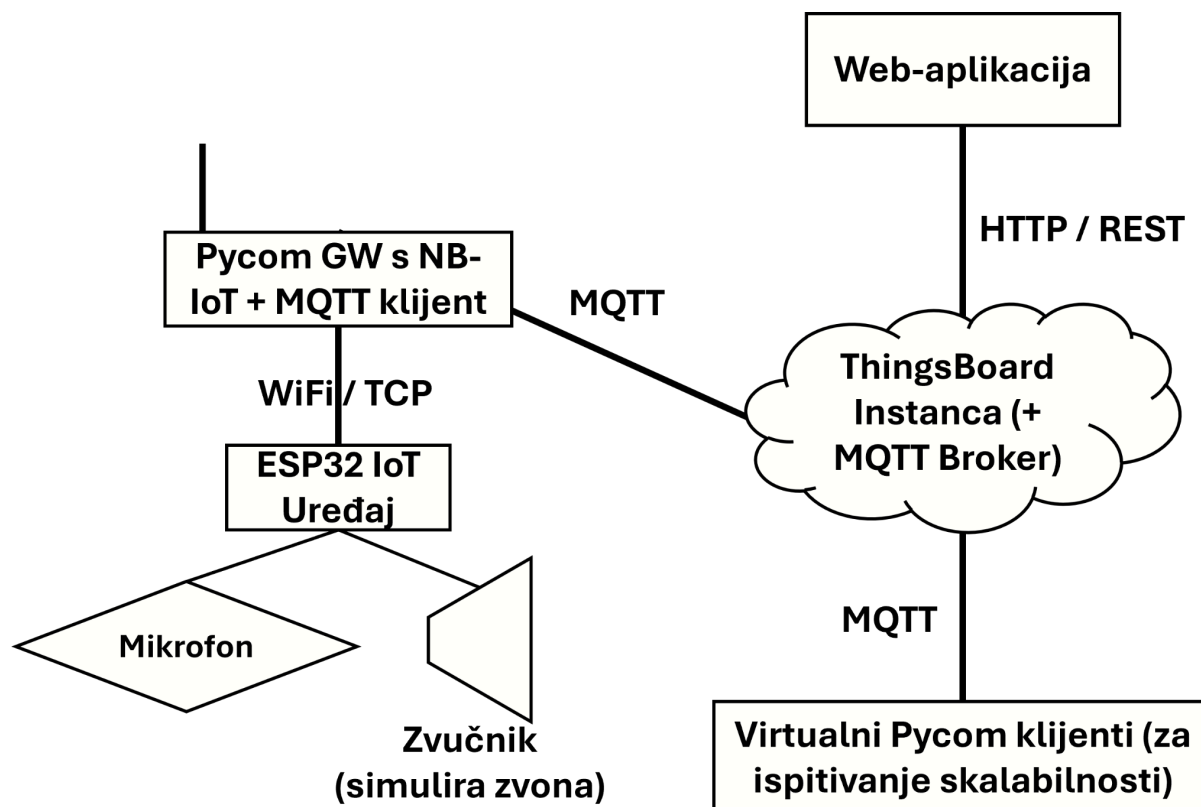
Korišteni su Pycom Gpy sa ekspanzijskom pločom, ESP32 mikrokontroler, kao senzor ICS-43434 mikrofona, a kao aktuator, za simulaciju zvona, MAX98357A pojačalo i zvučnik te IoT platforma ThingsBoard.

Slučajevi uporabe: Udaljeno upravljanje obrascima i glasnoćom zvonjenja, prikaz podataka o trenutnoj i povijesnoj razini buke unutar crkve, glasnoćom zvonjenja ovisno o trenutnom broju ljudi u prostoriji

Slični projekti: [SmartCustos.de](https://smartcustos.de), [ChimeMaster.com](https://chime-master.com), madonnapulsator.com,

2. Opis rješenja

ESP32 služi kao senzor i aktuator. Uzima glasnoću iz ICS-43434 mikrofona i simulira aktuator zvana pomoću MAX98357A pojačala i zvučnika, te sadržava svu logiku vezanu za praćenje rada zvona. ESP32 šalje i prima podatke TCP vezom od Pycom Gpy koji služi kao gateway čvor. Pycom je povezan na internet pomoću NB-IoT veze i razmjenjuje podatke pomoću protokola MQTT sa instancom ThingsBoard koja služi kao IoT platforma. React Web-aplikacija komunicira sa ThingsBoard instancom preko REST API-a.



Koristili su se ICS-43434 mikrofoni kao senzori i MAX98357A pojačalo i zvučnik kao aktuatori. Nasumične vrijednosti su se simulirale pomoću sin funkcije i dodanog stohastičkog šuma. Uz to imaju 2 vrijednosti zvuka “loud” i “soft”. Za komunikaciju između Pycoma i ThingsBoarda se koristio ugrađeni klijent, a između virtualnog Pycoma i TB-a klijent Paho.

2.1 ESP32 - Implementacijska dokumentacija

Uloga u sustavu

ESP32 djeluje kao senzorsko-aktuatorski čvor. Prikuplja podatke o razini buke u okolini te simulira aktuator zvona (oglašavanje zvučnih obrazaca). Sva logika vezana za praćenje i sviranje zvona izvodi se lokalno na ESP32, bez oslanjanja na vanjski poslužitelj. Podaci se razmjenjuju s Pycom GPy gateway čvorom putem TCP veze.

Senzori i aktuatori

1. Mikrofon - ICS-43434 (fizički senzor)

Tip: MEMS digitalni mikrofoni s I2S izlazom.

Spoj: I2S sučelje (BCLK = GPIO6, WS = GPIO7, DIN = GPIO5).

Način rada: ESP32 čita 32-bitne uzorke iz mikrofona pri frekvenciji uzorkovanja 16 000 Hz. Svake minute prikuplja 1 sekundu audio signala (16 000 uzoraka), računa RMS vrijednost i pretvara je u dB:

$$\text{dB} = 20 * \log_{10}(\text{RMS})$$

Kalibracija: pri pokretanju sustava izmjeri se referentna razina buke u tišini. Svako sljedeće mjerenje umanjuje se za tu referentnu vrijednost, čime se dobiva relativna promjena glasnoće u odnosu na početno stanje. Rezultat (jedan float, npr. "12.31") šalje se Pycomu porukom `/bell/loudness`.

2. Zvučnik s pojačalom - MAX98357A (simulirani aktuator)

Tip: I2S digitalni pojačavač s ugrađenim DAC-om.

Spoj: I2S sučelje (BCLK = GPIO6, WS = GPIO7, DOUT = GPIO4).

Način simulacije: stvarna crkvena zvona nisu priključena na sustav. Umjesto toga, ESP32 sintetizira sinusoidalne tonove odgovarajućih frekvencija i reproducira ih kroz zvučnik spojen na MAX98357A. Frekvencije odgovaraju glazbenim tonovima *chime* obrasca:

Ton 1 - 246 Hz

Ton 2 - 329 Hz

Ton 3 - 370 Hz

Ton 4 - 415 Hz

Svaki ton traje 500 ms, a pauze između fraza također traju 500 ms.

Obrasci zvonjenja definirani su kao nizovi tonova (1-4) i pauza (0):

:15 min - 4 3 2 1

:30 min - 2 4 3 1 | 2 3 4 2

:45 min - 4 2 3 1 | 1 3 4 2 | 4 3 2 1

:00 min - 2 4 3 1 | 2 3 4 2 | 4 2 3 1 | 2 3 4 2 | H * (ton 1)

gdje H odgovara satu u 12-satnom formatu. Obrasci se mogu dinamički mijenjati naredbom `/bell/pattern` i vratiti na zadane vrijednosti naredbom `/bell/reset`.

Komunikacija između komponenti

ESP32 spaja se kao Wi-Fi STA klijent na pristupnu točku Pycom GPy uređaja (SSID: "PYCOM-ECRKVA"). Nakon uspostave veze otvara se TCP klijentska konekcija prema poslužitelju na adresi 192.168.4.1, port 9000.

Protokol: tekstualni. Svaka poruka je jedna UTF-8 linija završena znakom `'\n'`. Format poruke: `"<naredba> [argument]"`.

Poruke koje ESP32 šalje Pycomu:

`/bell/loudness <float>` - izmjerena relativna glasnoća u dB (jednom u minuti)

Poruke koje ESP32 prima od Pycom:

/bell/ring <1-4> - odsviraj jedan ton

/bell/pattern <min> <t1> <t2> ... - postavi novi obrazac za dani slot (0/15/30/45)

/bell/reset - vrati sve obrasce na defaultne vrijednosti

Upravljanje vezom: rx_task FreeRTOS zadatak neprekidno čita znakove s TCP socketa i akumulira ih u line buffer. Pri gubitku veze (r == 0 ili errno != EAGAIN) socket se zatvara i automatski se pokušava ponovno spajanje nakon 3 s.

Sinkronizacija vremena: pri pokretanju ESP32 sinkronizira sat putem SNTP protokola (pool.ntp.org). Postavljeno je hrvatsko vremensko područje (CET/CEST). Zvonjenje se ne izvodi dok sat nije sinkroniziran.

Razvojni okviri i biblioteke

1. ESP-IDF (Espressif IoT Development Framework) - osnovna razvojna platforma za ESP32.

Koriste se sljedeće komponente:

driver/i2s_std - I2S komunikacija s mikrofonom (RX) i pojačalom (TX)

esp_wifi - Wi-Fi STA mod i upravljanje događajima

esp_netif_sntp - SNTP sinkronizacija vremena

nvs_flash - inicijalizacija NVS particije (potrebna za Wi-Fi stog)

2. FreeRTOS (ugrađen u ESP-IDF) - višezadaćnost:

bell_task (prioritet 4) - provjera trenutnog vremena svakih 20 s, pokretanje odgovarajućeg obrasca zvonjenja

loudness_task (prioritet 3) - mjerenje glasnoće i slanje jednom u minuti

net_rx task (prioritet 5) - primanje TCP poruka i pozivanje callback handlera

3. IoT platforma

Za realizaciju sustava korištena je ThingsBoard IoT platforma. Riječ je o open-source platformi koja se koristi za povezivanje i upravljanje IoT uređajima, prikupljanje podataka, obradu telemetrije te prikaz podataka putem vizualizacija. ThingsBoard podržava uobičajene IoT komunikacijske protokole kao što su MQTT, CoAP i HTTP. Platforma se može koristiti u oblaku, ali i lokalno, odnosno kao on-premise rješenje.

U ovom projektu ThingsBoard je korišten kao centralna platforma za registraciju uređaja, organizaciju entiteta, zaprimanje telemetrije, obradu podataka putem rule chain-ova i prikaz prikupljenih podataka na dashboardu.

ThingsBoard omogućuje izradu IoT sustava pomoću različitih entiteta kao što su uređaji, asseti, profili, veze, rule chain-ovi i dashboardi. Uređaji predstavljaju IoT komponente koje šalju telemetrijske podatke, dok se asseti koriste za prikaz lokacija, objekata ili organizacijskih cjelina te za povezivanje pripadajućih uređaja.

Platforma koristi koncept digitalnog blizanca, gdje su stvarni objekti prikazani kao povezani entiteti s atributima, podacima i alarmima. Rule chain-ovi služe za obradu podataka koji dolaze s uređaja, primjerice za filtriranje, transformaciju i pokretanje alarma. Za prikaz podataka koriste se dashboardi, koji omogućuju pregled trenutnih i povijesnih vrijednosti pomoću grafova, tablica, kartica i drugih widgeta.

ThingsBoard se može koristiti kroz više različitih opcija. Dostupna je Community Edition verzija, koja je open-source, zatim Professional Edition s naprednijim mogućnostima te ThingsBoard Cloud kao cloud usluga.

ThingsBoard Cloud je upravljana platforma za IoT aplikacije, a naplata ovisi o odabranom planu, broju uređaja i količini poslanih podataka. Platforma nudi i besplatno probno razdoblje od 30 dana, što je korisno za testiranje osnovnih funkcionalnosti.

Za potrebe projekta ThingsBoard se može koristiti kroz probnu cloud verziju ili kao self-hosted rješenje. Kod self-hosted Community Edition verzije nema pretplate, ali je potrebno osigurati vlastitu infrastrukturu, hosting i održavanje.

Struktura sustava

Na ThingsBoard platformi definirana su dva glavna asseta: Crkva i Zvonik. Asset Crkva predstavlja nadređeni objekt, odnosno lokaciju sustava, dok asset Zvonik predstavlja dio crkve u kojem se nalaze povezani IoT uređaji.

Zvonik je povezan s Crkvom kroz relaciju pripadnosti, čime se dobiva jasna struktura sustava. Uređaji su povezani sa Zvonikom kako bi se prikupljena telemetrija mogla logički povezati s mjestom na kojem nastaje.

Korišteni uređaji i komunikacija

Glavni uređaj u sustavu je ESP32, na koji su lokalno povezani mikrofoni i zvono/zvučnik. Te komponente nisu zasebno definirane kao uređaji u ThingsBoardu, nego se koriste kao periferija ESP32 uređaja. ThingsBoard zato ne komunicira izravno s njima, već prima podatke koje ESP32 prikupi i pošalje.

ESP32 je povezan s Pycom uređajem, koji u sustavu služi kao gateway prema ThingsBoard platformi. Pycom proslijeđuje podatke korištenjem MQTT komunikacije, pri čemu se telemetrija šalje prema ThingsBoardu kroz MQTT poruke. Na taj način platforma može zaprimati podatke s uređaja, a po potrebi podržava i attribute, RPC komunikaciju te provisioning uređaja.

Device provisioning

Na platformi je omogućena funkcionalnost device provisioning, koja služi za automatsku registraciju uređaja. Time nije potrebno svaki uređaj ručno unaprijed kreirati u korisničkom sučelju. Uređaj se pri prvom spajanju prijavljuje platformi, a ThingsBoard ga nakon provjere registrira i dodjeljuje mu potrebne pristupne podatke.

U ovom projektu device provisioning se koristi kako bi se ESP32/Pycom dio sustava mogao jednostavnije i kontrolirano povezati s platformom. Na taj način olakšano je dodavanje uređaja te njihovo povezivanje s odgovarajućim profilima i rule chain-ovima.

Device profile, asset profile, rule chain, dashboard

Za uređaje su definirani device profile-i, koji određuju osnovno ponašanje uređaja na platformi i povezuju ih s odgovarajućim rule chain-om. Time se uređaji iste vrste mogu jednostavnije konfigurirati, bez ručnog podešavanja svakog uređaja posebno.

Za organizaciju asseta, kao što su crkva i zvonik, korišteni su asset profile-i. Oni omogućuju preglednije grupiranje objekata i stvaranje jasne hijerarhije sustava kroz relacije između asseta i uređaja.

Obrada podataka odvija se pomoću rule chain-ova. Oni primaju telemetriju s uređaja, obrađuju je i spremaju kako bi se kasnije mogla prikazati na dashboardu. U ovom projektu rule chain-ovi se koriste za obradu podataka poslanih preko MQTT komunikacije.

Za prikaz prikupljenih podataka koristi se ThingsBoard dashboard. Dashboard omogućuje prikaz trenutnih i povijesnih vrijednosti te grafički pregled promjena kroz vrijeme. Na taj način moguće je pratiti stanje sustava preko web sučelja, bez potrebe za izravnim pristupom ESP32 uređaju.

Konfiguracija resursa

Resursi spojeni na platformu konfigurirani su kroz ThingsBoard uređaje, device profile i rule chainove. U sustavu su definirana dva tipa uređaja: gateway uređaj temeljen na Pycom platformi i uređaj za upravljanje zvonom temeljen na ESP32 mikrokontroleru.

Za gateway uređaj definiran je odgovarajući device profile koji omogućuje MQTT povezivanje s ThingsBoard platformom te razmjenu telemetrije i RPC poruka. Gateway je zadužen za komunikaciju prema platformi i proslijeđivanje podataka s povezanih uređaja.

Za ESP32 uređaj definiran je zaseban device profile koji predstavlja čvor u zvoniku na koji su spojeni senzor i aktuator. Na ESP32 je spojen mikrofonski senzor za mjerenje glasnoće te aktuator za upravljanje zvonom. ESP32 se ne povezuje direktno na ThingsBoard, nego ga gateway prijavljuje kao downstream uređaj preko ThingsBoard Gateway MQTT API-ja.

Rule chainovi konfigurirani su za obradu pristigle telemetrije, generiranje alarma i obradu RPC naredbi. Telemetrija mikrofona koristi se za praćenje razine glasnoće i detekciju

neobičajenih stanja, dok se naredbe za upravljanje zvonom prosljeđuju prema odgovarajućem uređaju.

```
15
16 TB_MQTT_HOST = os.getenv("TB_MQTT_HOST", "161.53.133.253").strip()
17 TB_MQTT_PORT = int(os.getenv("TB_MQTT_PORT", "1883"))
18
19 TB_PYCOM_DEVICE_NAME = os.getenv("TB_PYCOM_DEVICE_NAME", "pycom-gateway-001").strip()
20 TB_ESP_DEVICE_NAME = os.getenv("TB_ESP_DEVICE_NAME", "esp32-bell-node-001").strip()
21
22 TB_PYCOM_TOKEN_FILE = os.getenv("TB_PYCOM_TOKEN_FILE", ".tb_pycom_gateway_token").strip()
23
24 PROVISION_REQUEST_TOPIC = "/provision/request"
25 PROVISION_RESPONSE_TOPIC = "/provision/response"
26 PROVISION_USERNAME = "provision"
27 PROVISION_TIMEOUT_S = 20
28
29 PYCOM_TELEMETRY_TOPIC = "v1/devices/me/telemetry"
30 PYCOM_ATTRIBUTES_TOPIC = "v1/devices/me/attributes"
31
32 GATEWAY_CONNECT_TOPIC = "v1/gateway/connect"
33 GATEWAY_DISCONNECT_TOPIC = "v1/gateway/disconnect"
34 GATEWAY_TELEMETRY_TOPIC = "v1/gateway/telemetry"
35 GATEWAY_ATTRIBUTES_TOPIC = "v1/gateway/attributes"
36 GATEWAY_RPC_TOPIC = "v1/gateway/rpc"
37
38 # Fallback/direct Pycom RPC topics.
39 # Use these if ThingsBoard gateway RPC to the ESP32 downstream device times out.
40 DIRECT_RPC_REQUEST_TOPIC = "v1/devices/me/rpc/request/+"
41 DIRECT_RPC_RESPONSE_TOPIC_PREFIX = "v1/devices/me/rpc/response/"
42
```

Na slici se vidi osnovna konfiguracija ThingsBoard uređaja i topica.

```
48
49 PYCOM_STATIC_ATTRIBUTES = {
50     "device_role": "lte_gateway",
51     "model": "Pycom",
52     "firmware_version": "1.0.0",
53     "location": "Crkva - zvonik",
54 }
55
56 ESP_STATIC_ATTRIBUTES = {
57     "device_role": "bell_node",
58     "model": "ESP32",
59     "location": "Crkva - zvonik",
60     "has_microphone": True,
61     "has_bell_actuator": True,
62     "microphone_unit": "dBA",
63     "warning_threshold_db": 70,
64     "critical_threshold_db": 85,
65 }
```

Ovdje definiramo attribute uređaja. Pycom koristimo kao gateway, a esp32 kao spojnicu između mikrofona i zvona te pycoma.

```
296
2+ usages  Mmacukic
297 def gateway_connect_esp(client: mqtt.Client) -> None:
298     payload = {
299         "device": TB_ESP_DEVICE_NAME
300     }
301
302     publish_json(client, GATEWAY_CONNECT_TOPIC, payload, qos=0)
303     print(f"Gateway announced downstream ESP32 device: {TB_ESP_DEVICE_NAME}")
304
305     # Give ThingsBoard a moment to register/update the downstream gateway session.
306     time.sleep(2)
307
308
309 1 usage  Mmacukic
309 def gateway_disconnect_esp(client: mqtt.Client) -> None:
310     payload = {
311         "device": TB_ESP_DEVICE_NAME
312     }
313
314     publish_json(client, GATEWAY_DISCONNECT_TOPIC, payload, qos=0)
315     print(f"Gateway disconnected downstream ESP32 device: {TB_ESP_DEVICE_NAME}")
316
```

Slika prikazuje prijavu esp32 kao downstream device od strane pycom uređaja.

4. Korisničke aplikacije

4.1. Opis i namjena aplikacije

Za potrebe krajnjih korisnika (npr. župnika, crkvenih administratora ili tehničkog osoblja) razvijena je namjenska web-aplikacija koja služi kao centralno sučelje za nadzor i upravljanje crkvenim zvonikom. Primarna namjena aplikacije je omogućiti udaljeni uvid u stvarno stanje na terenu, praćenje glasnoće i ispravnosti rada zvona te izravno slanje upravljačkih naredbi (RPC) bez potrebe za kompleksnim, izravnim pristupanjem ThingsBoard administracijskom sučelju.

4.2. Korištene tehnologije

Aplikacija je realizirana kroz klijent-poslužiteljsku arhitekturu koja se sastoji od sljedećih tehnologija:

Frontend (Klijentski dio): Razvijen je pomoću programskog okvira React. Koristi se za izgradnju dinamičnog i responzivnog korisničkog sučelja.

Backend (Poslužiteljski dio): Realiziran je pomoću Node.js okruženja i Express radnog okvira. Za razliku od izravne komunikacije klijenta s IoT platformom, backend je u ovom sustavu primarno implementiran kako bi služio kao trajni repozitorij podataka i posrednik (proxy). Njegova glavna zadaća je automatizirano prikupljanje i spremanje zapisa o zvonjavama (povijest glasnoće u decibelima i točno vrijeme sinkronizacije) u lokalnu bazu podataka, što se izvršava periodički svakih 15 minuta.

Komunikacija i pohrana podataka: Komunikacija s ThingsBoard platformom u potpunosti se oslanja na ThingsBoard REST API preko kojeg backend povlači telemetriju i prosljeđuje RPC naredbe. Autentifikacija je riješena dinamički kroz prijavu korisnika i generiranje JWT tokena. Podaci koje backend periodički prikuplja trajno se strukturiraju i pohranjuju unutar pripadajuće baze podataka, čime se rasterećuje sama IoT platforma i omogućuje brži povijesni pregled na klijentskoj strani.

4.3. Implementacijski detalji i integracija s ThingsBoard REST API-jem

Komunikacijski protokol i životni vijek podataka unutar aplikacije odvijaju se kroz četiri ključna koraka integrirana na backend razini:

Autentifikacija korisnika:

Prilikom pokretanja aplikacije šalje se POST zahtjev na endpoint `/api/auth/login` s korisničkim podacima. Iz odgovora platforme backend preuzima i pohranjuje `JWT_TOKEN` i `REFRESH_TOKEN` koji se koriste kao autorizacijsko zaglavlje (X-Authorization: Bearer) za sve daljnje upite.

Dohvat identifikatora uređaja:

Sustav šalje GET zahtjev na `/api/tenant/devices` kako bi preuzeo popis svih registriranih uređaja i njihovih jedinstvenih `deviceId` parametara.

Dohvat telemetrijskih podataka:

Podaci o trenutnom stanju senzora čitaju se putem GET zahtjeva na endpoint `/api/plugins/telemetry/DEVICE/{deviceId}/values/timeseries`. Backend s ThingsBoard platforme eksplicitno povlači isključivo trenutne (posljednje zabilježene) vrijednosti telemetrije za ključeve: `microphone_loudness`, `bell_state`, `last_pattern`, `last_ring_time`, `esp_online` i `lte_signal`. Ove trenutne vrijednosti se potom prikazuju na sučelju, ali se paralelno (svakih 15 minuta) trajno zapisuju u lokalnu bazu podataka na backendu kako bi se formirao povijesni niz potreban za generiranje grafikona.

Slanje upravljačkih naredbi (RPC):

Slanje jednosmjernih naredbi prema aktuatorima izvršava se putem POST zahtjeva na `/api/plugins/rpc/oneway/{deviceId}`, gdje se u tijelu poruke šalju metode poput `bell.ring`, `bell.reset` ili odabrani `bell.pattern`.

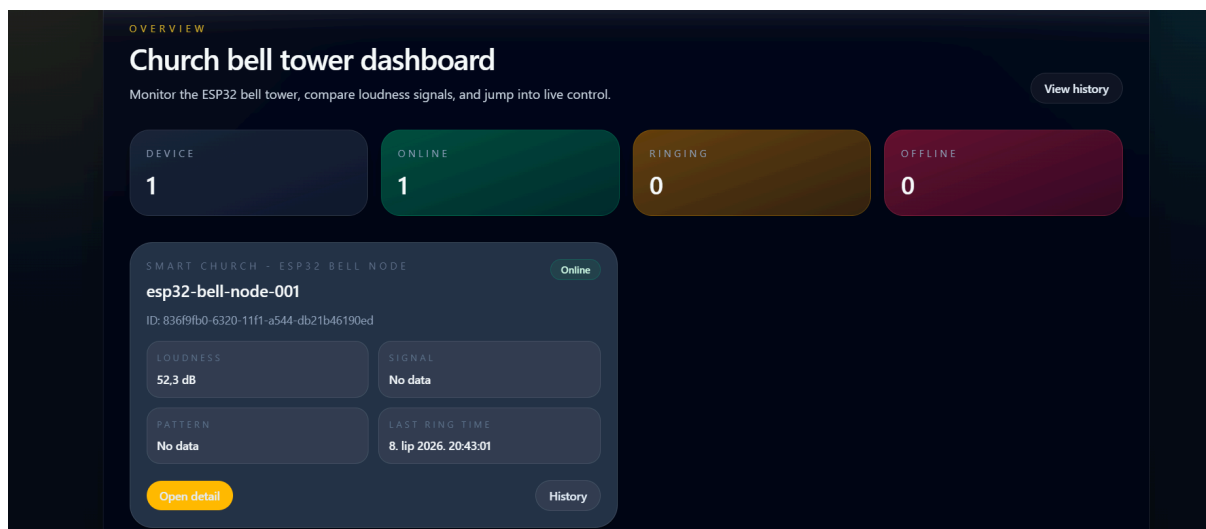
4.4. Izgled i funkcionalnosti korisničkog sučelja

Aplikacija je strukturirana kroz tri glavna ekrana: **Glavna stranica (Dashboard Overview)**, **Detalji uređaja (Device Details)** i **Povijest (Microphone Loudness History)**.

4.4.1. Glavna stranica (Dashboard Overview)

Gornji dio glavne stranice sadrži zbirnu KPI traku koja prikazuje statistiku cijelog sustava (ukupan broj uređaja, broj online/offline čvorova te broj zvona koja trenutno aktivno zvone). Iako je u trenutačnoj izvedbi sustav testiran s jednim aktivnim čvorom (`esp32-bell-node-001`), dizajn je u potpunosti skalabilan te bi u slučaju proširenja na više crkava ili zvonika automatski prikazivao zbirne podatke.

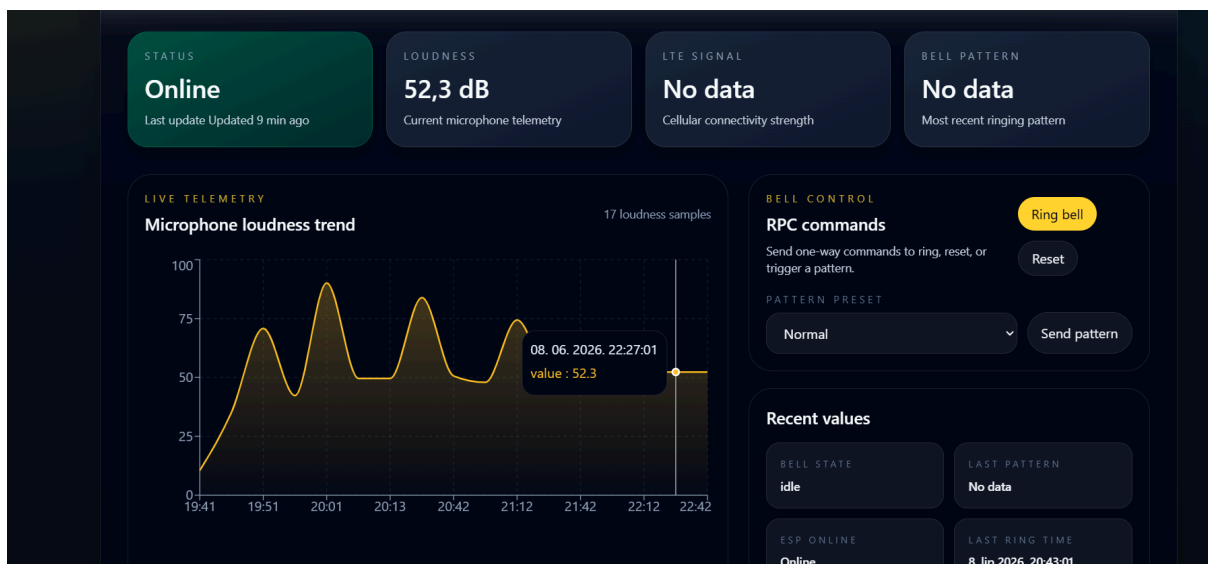
Ispod statistike nalazi se lista uređaja. Za svaki uređaj prikazana je kompaktna kartica s ključnim informacijama: jedinstveni ID, mrežni status (Online), zadnja izmjerena glasnoća u decibelima, vrijeme zadnjeg zvonjenja. Iz ove kartice korisnik može navigirati u dublju analizu klikom na gumb *Open detail* ili *History*.



4.4.2. Detalji uređaja (Device Details)

Klikom na *Open detail* otvara se upravljačka ploča pojedinog zvonika. Ovaj ekran podijeljen je na tri logičke cjeline:

- **Statusne kartice:** Prošireni prikaz mrežnog statusa, trenutne telemetrije mikrofona i trenutno aktivnog obrasca zvonjenja.
- **Vremenski graf (Live Telemetry):** Interaktivni grafikon koji prikazuje trend kretanja glasnoće u crkvi (Microphone loudness trend) unutar **zadnjih 6 sati**. Grafikon jasno vizualizira skokove u glasnoći koji se događaju prilikom aktivacije zvona, omogućujući administratoru vizualnu potvrdu da zvono fizički radi.
- **Upravljačka ploča (Bell Control):** Sučelje za slanje RPC naredbi u stvarnom vremenu. Korisnik može trenutačno aktivirati zvono klikom na gumb *Ring bell*, resetirati rad kontrolera preko gumba *Reset*, ili iz padajućeg izbornika odabrati i poslati željeni predefinirani obrazac zvonjave (*Pattern Preset* - npr. *Normal*).



4.4.3. Povijest (Microphone Loudness History)

Ekran *History* namijenjen je dugoročnijoj analizi rada sustava. Sadrži prošireni vremenski grafikoni koji obuhvaća podatke o glasnoći unutar **zadnja 24 sata**. Ova funkcionalnost omogućuje povijesni pregled svih aktivacija zvona tijekom dana i noći, detekciju eventualnih anomalija u radu ili uvid u razinu buke unutar same crkve kada zvona nisu aktivna (npr. detekcija prisutnosti ljudi ili buke iz okoline). Iz ovog prikaza korisnik se jednim klikom na gumb *Open device detail* može vratiti natrag na kontrolnu ploču.



